

Lab 4 Report

Authors:

Steven Gong

Nicholas Leung

Bobby Taing

Date: 12/5/2025

Table of Contents

Challenge 1	2
Approach and Strategy	2
Code Implementation.....	2
Design Rationale.....	4
Problem-Solving Process.....	4
Performance Analysis	5
Challenge 2	6
Approach and Strategy	6
Code Implementation.....	6
Design Rationale.....	7
Problem-Solving Process.....	7
Performance Analysis	8
General Analysis	9
Overall Reflection.....	9
Course Content Application	9
Process vs. Outcome.....	10
Lessons Learned	10
References	11
Hours Spent & Feedback	11

Challenge 1

Approach and Strategy

This challenge requires precise peg placement, with small margins of error. Our approach to this challenge was to ensure consistency with how the robot moved and placed pegs. We needed to secure the wooden bench to keep our target locations the same at all times. We also needed a constant location to pick up pegs from. Additionally, the way we place pegs matters. Placing pegs in an order that prevents interference with future peg placements means we can place pegs using consistent movements: Hover a few centimeters above the target, then move down into the slot.

We noticed that the bench's slots had plastic inserts that didn't allow the pegs to slide in naturally. This left us with two options: force a substantial part of the peg through the insert and into the slot, or place the peg straight into the slot so it will rest on top of the insert. One approach requires more force from the robot arm but will guarantee more consistent results, and the other option has less complicated movements but has a higher chance of going into the slot crooked. We decided to prioritize the first approach because we can use human interventions if needed to secure the pegs.

Once we determined our set locations for pickup and placement, we mainly drew upon Lab 2 for strategies. Drag teaching the robot to our desired locations gave us consistent coordinates to work with. We also used forward and inverse kinematics to make sure our robot calculates consistent joint angles to reach the same coordinates. We recycled kinematics code from previous labs to help us develop our code faster and get the robot arm moving earlier on in the lab.

When we first started this lab, we didn't have access to the pegs. It was also difficult for all three of us to meet in lab at the same time. To get around this issue, we worked on Challenges 1 and 2 in parallel.

Code Implementation

Our code is organized into key modules and control tasks that work together to operate the MechArm 270 Pi robot and enable it to complete the required action. The way we structured the code will minimize code length, make function reusable, and make the system easier to debug and adjust; we share the helper functions between Challenge 1 and Challenge 2.

For both challenges, we learn the desired position that we need the robot to be at by drag teaching. With `displayRobotStatus()` and the `Read10Position()` function from Lab2, we recorded the coordinates and angles needed to perform tasks for challenge 1 and 2. The function used were `get_coords()` and `get_angles()` function from the MyCobot Python API to get the robot arm's position.

```

# Display the current end effector coordinates and the corresponding joint angles
def displayRobotStatus(mycobot: MyCobot):
    angles = mycobot.get_angles()
    coord = mycobot.get_coords()
    # Odd Line will be angles while Even will be coords
    print(angles)
    print(coord)

```

Figure 1. Helper function to return the current angles and coordinates of the robot.

Our group chose to use the same method as Lab3 to control the arm in challenge 1 and 2: using inverse kinematics and then forward kinematics. To control the robot's motion, we implemented a pipeline that computes joint angles using inverse kinematics and then compute for the final coordinates using forward kinematics with the computed joint angles. It'll ensure the computed angles from inverse kinematics bring us to the same coordinates during drag teaching. We created a reusable function getCoordinates(coords, angles) to accomplish the action. It takes in the recorded coordinates and angles and return the computed coordinate in an array "[x, y, z, roll, pitch, yaw]" that will be sent to the arm.

```

#Performs inverse kinematics to get solution joint angles, and recalculates solution coordinates using our own forward kinematics
def getCoordinates(coords, angles):
    x_target = coords[0]
    y_target = coords[1]
    z_target = coords[2]
    rx_d = np.radians(coords[3]) # Roll angle (in radians)
    ry_d = np.radians(coords[4]) # Pitch angle (in radians)
    rz_d = np.radians(coords[5]) # Yaw angle (in radians)
    q_init = angles
    joint_angles = ik((x_target, y_target, z_target, rx_d, ry_d, rz_d), q_init) # apply inverse kinematic to find computed angles
    calculated_coords = symbolic_forward_kinematics(joint_angles) # apply forward kinematics to find the computed coordinates with computed angles
    # Since the coordinates that we got only contains x y z... we then add the "recorded" rx ry rz
    end_effector_position = calculated_coords[:3, 3] + [coords[3], coords[4], coords[5]]
    # Send coordinates to robot
    return end_effector_position

```

Figure 2. Helper function to get computed coordinates with given coordinates and angles that recorded by drag teaching.

For other API usage, we used send_coords() to control the arm moving to the coordinates returned by the getCoordinates() function. We also used send_angles() to set the arm to neutral position. We used the set_gripper_value() and set_gripper_state() function to control the gripper's motion to close and open. Also, we used the NumPy and Sympy libraries to calculate matrixes symbolically or numerically. Additionally, we used least_squares() from Scipy to approximate inverse kinematics solutions, ensuring that the computed joint angles matched the desired end-effector position as closely as possible.

In Challenge 1, the arm's motion is controlled by the the_Challenge() function. This function move the arm to the neutral position, then executes three sequential of pick and place operations by calling the pickUpPeg() and placePeg() functions. After completing all three sequences, the robot returns to the neutral position. In the peg challenge, we only use one loading zone where the pickup coordinates are the same for all pegs. To make the process efficient, we implemented the pickUpPeg() function, which asks the arm to pick up the peg at the loading zone and then returns the arm to neutral. The placePeg() function accepts two coordinate inputs: a via point used to safely approach the target area and the final coordinate

where the peg is placed. The function moves the arm to the via point, then to the final placement coordinate, and once the peg is placed, the arm returns to the via point before moving back to neutral.

Design Rationale

We decided to go for three pegs only. We successfully completed Challenge 2 before challenge 1, so we did not need bonus points as insurance points for Challenge 2. We oriented the bench to have the row of yellow, orange, red slots closer to the robot arm so we have enough space on the left side of the board for the end effector to reach. We planned to place pegs in the three closest slots so the robot joints can stay within their angle limits. This would also give the robot the most options for valid joint angles when performing inverse kinematics, so it will eventually converge on a good solution. Our constant pickup location was to the right of the bench, so it wouldn't block the arm as it moved to pick up the peg.

We mounted the gripper to point down towards the wooden bench. This is because we wanted the arm to move in the same direction that the peg is pointing when placing them. Drag teaching was easier to perform and the gripper can use a gravity assist to push the peg down into the slot.

Our robot movements involved one via point before grabbing or releasing the peg. This prevents the robot arm from knocking over the peg before it moves into position. As discussed in lecture, having more via points gives us more finer control over how the robot moves in space, since the arm needs to move through the via points. Between picking up and placing a peg, the arm moves back to the neutral position. This acts as a way for us to "calibrate" the arm back to a known reference position before moving again. More calibrations leads to more consistent movements.

Another decision we made for our robot movements was having it move at 30% speed or less. After reaching the via point, we set the speed to 10%. We also incorporated plenty of `sleep()` calls between movements. This gives the arm enough time to settle on the correct position, as well as helping to push the peg into the slot beyond the plastic insert. Slow, steady movements prevents the pegs from falling over.

When determining our target coordinates, we gathered initial coordinates from drag teaching. Similar to Lab 2, this gives us a good set of coordinates to start with and improve without needing to do a lot of guess-and-checking.

Problem-Solving Process

One challenge we ran into was picking up the pegs properly. When we set the gripper state to 1, the gripper struggled to properly grab the peg and sometimes it crashed the robot arm. We needed to grab the peg firmly enough so it can move locations but not so tightly where the gripper servo strains itself. After some guidance from Tawsif and the `pymycobot` API reference, we changed our function call for closing the gripper to `send_gripper_value()`. This let us partially close the gripper and put some compression on the peg. This change was successful because the peg never slipped and the robot stopped crashing.

There were also times when the gripper never opens after placing the peg, despite us calling `set_gripper_state()`. To solve this issue, we commanded the gripper to open at each via point on the way to the neutral position. This represents a benefit of using via points in our trajectories.

Another challenge we encountered was drag teaching being inaccurate at times. When we perform inverse kinematics on our drag-taught coordinates and move the robot, it's final position would be different from drag teaching. If the robot movements fail, we make slight adjustments to the target X, Y, and Z values in the CSV file. Making small changes to the coordinates one axis at a time allowed us to finely tune where the robot needs to move. Once we can get one peg down, the remaining slots were just a matter of applying an offset in the X and Y directions relative to the first peg. We also adjusted our inverse kinematics code to use the robot's current joint angles as the initial guess. This minimizes the chance of creating robot configurations with awkward joint angles.

One more challenge we encountered was the last peg not moving into place properly. Our initial plan was to place the yellow, orange, and red pegs from left to right. However, the red peg was too close to the edge of the bench, and it blocked the end effector from moving the peg all the way down into the slot. To solve this problem, we changed the peg placement sequence to yellow, then light green in the back row, and then orange. Since the green peg is a little more to the left than the red peg and not too far forward from the robot arm's base, we were able to quickly drag teach a new set of coordinates. We also placed the light green peg second because it will be out of the way when it's time to place the orange peg. Small changes to our CSV file let us change the place order quickly, showing that our code to parse the CSV file and turn it into robot movements is well-optimized. As a result, we successfully placed three pegs in the manner we intended.

Performance Analysis

The robot performed well in this challenge. It successfully picked up three pegs and placed them into their respective slots. The robot's movements matched our expectations. Trajectories were consistent throughout multiple runs and the pegs always landed on target. We did need to use two human interventions to get the last two pegs fully in the slot, but this still satisfies the requirements of the challenge.

The time it took for our robot arm to complete the challenge was about two and half minutes. This also matches our expectation for slow, consistent, and accurate movements. If we were to do this challenge again, we would see where we could reduce sleep times to make the robot move faster but with the same level of precision and accuracy. In our code, the longest sleep time was seven seconds, with most sleeps being two to three seconds. Cutting down on some of these sleep times can reduce the challenge completion time. Alternatively, increasing the speed of our robot arm could also reduce completion time.

Our use of via points before picking up and placing pegs was also very successful. Hovering above and moving straight down then up erased our concerns about interfering with other pegs or obstacles.

Challenge 2

Approach and Strategy

At the beginning of the project, we thought the challenge will be just an easy pick and drop task that can be solved with the same inverse and forward kinematic pipeline developed in Labs 3. Based on this assumption, our initial strategy was to drag teach the robot arm and record the record angles and end-effector coordinates for each block we planned to pick up and place; then, we compute for coordinates using inverse kinematics and then forward kinematics. Considering the previous lab (Lab 2's Challenge about drawing dots on the board) regarding the accuracy of the drag teaches and path planning as it moves to a coordinate, we came up with two conclusions: (1) drag teaching can't get us the an actual coordinates but the approximate one to pick up/drop the block and the (2) route where arm is taking to approach the desired coordinates might knock off the tower.

Due to the challenges, we must approach the desired coordinates by manually changing the number in the recorded coordinates until the end effector go to the coordinates that we expected. In addition, we added multiple via point where arm must approach before it get closer to a block and perform action to pick up/drop, this approach forces the arm to move in a path where it wouldn't hit the tower.

Once we get the best coordinates to pick up/drop the block and via point that forces the arm to take best path to approach the block, we just need to send those coordinates and via points into the inverse and forward kinematics pipeline developed in Lab3. Then, send the resulting coordinates to the robot again and again until it finishes the tasks of deconstructing and constructing a new tower. We continue to choose sending recorded coordinates to inverse kinematic and feed the result angles of inverse kinematic in forward kinematic to get computed coordinates. This strategy ensures we're calculating the best angles with inverse kinematic moving to the same coordinates.

Another strategy we decided on was reconstructing the new tower immediately after removing a block from the original tower. When the robot places a block down, that will be where the new tower is. This minimizes failure points because we reduce the number of moves the robot has to make.

Code Implementation

The technical aspects of code and how the forward and inverse kinematic was implemented were the same as Challenge 1. The strategy is the same as Challenge 1 too. In addition, the helper function was shared between Challenge 1 and 2.

In Challenge 2, the arm's motion is coordinated through the `sendRobotToSpecificLocation()` and `blockPickNDrop()` function. The `blockPickNDrop()` function will read the file and organize the data into list before moving the robot. The data in the csv file will be sort into four types of lists: a list of coordinates and angles for via point before picking up a block, a list of coordinates and angles for via point before dropping a block, and a list of coordinates for location where block will be pickup/place. The function then sends these

coordinate lists to `sendRobotToSpecificLocation()`. This function executes the sequence of movements required to pick up and place a block. The function is structured like a finite state machine. In State 1, the arm performs all actions needed to pick up the block and then returns to the neutral position. In States 2, the arm will place the block at the desired coordination and again returns to neutral. Overall, the function helps the arm to accomplish tasks for challenge 2.

Design Rationale

The two main issues that drove our design choice were: (1) the robot often moved to coordinates that were not identical to the drag teach values and (2) the robot repeatedly knocked down the tower while moving toward a desired coordinate.

We decided to mount the gripper to point downward. Although this configuration slightly reduces the maximum horizontal reach of the arm, it significantly reduces the complexity of orientation tuning during the task. When the gripper points downward, the tool frame aligns with the tower, and just needed to move the gripper downward. It allows the robot to pick blocks from the top of the tower without requiring additional wrist rotations.

We improved the pick and place position by manually adjusting the x, y, and z of the recorded coordinates while keeping the orientation (Rx, Ry, Rz) fixed. We chose to keep the original orientation because drag-teaching process already produced a reliable orientation for the gripper to hold the block. By adjusting only the position variables, we can slowly align the theoretical coordinates more closely with the actual coordinates that's best to pick up/drop the block. This process is repeated six times to tune six coordinates (three different coordinates to pick up the block and another three to drop off the block).

To prevent the arm from knocking down the tower, we redesigned the motion as a multi-step trajectory with via points, rather than a single direct move. This technique forces the arm to move in an expected path that would avoid obstacles. Thus, we add different via points before it will pick/drop any block. The sequence is now: (1) Go to neutral (once at the start), (2) Go to via point 1 (above the block), (3) Move down to the block coordinates, (4) Close Gripper to pick up block, (5) Return to via point 1, (6) Return to neutral, (7) Move to via point 2 (above the drop area), (8) Move down to the drop coordinates, (9) Open gripper to drop block, (10) Return to via point 2, and (11) Return to neutral. The sequence will repeat until it picks up the first, third, and fifth block of the tower and build another tower with it. With experimentation, this sequence perfectly avoids knocking down the tower and accomplishes the goal.

Problem-Solving Process

Right after we drag teach the robot to pick up three blocks and drop them at the desired locations, we wrote a main function to automate the sequence. The sequence was picking blocks from the loading zone and placing them in another zone to build the tower. During the test, we encountered three challenges: the robot often moved to coordinates that aren't what we taught the robot, the robot repeatedly knocked down the tower in the route moving to a desired coordinate, and the testing process was takes time because every change required rerunning the entire sequence.

We reorganized our code to make the function more reusable and easier for us to debug. Originally, we combine coordinate handling, inverse kinematics, and commands of robot into one large function, but now we separated the logic. We implemented a function that'll take in coordinates and angles and return computed coordinates: it calculates joint angles using inverse kinematics, then sends those angles through our forward kinematics to verify the resulting position. We also rewrote the main function as a finite state machine. In states 1, the robot executes the pickup sequence and returns to neutral. In states 2, it executes the drop sequence and returns to neutral. The finite state machine cycles through these states until the list of coordinates runs out. This structure made the control flow clearer which allowed us to learn at what stages the robot goes out of control.

We improved the positional accuracy by manually refining the recorded drag-teach coordinates. We then created a utility function that sent the robot from neutral to via point to pickup/drop coordinate. Then we iteratively edited the coordinate values in code and reran just this test routine until the end effector can reach a position that was close enough to the desired location. This solution indeed minimizes the error in picking up the block and dropping it in the right place, but there are still small placement errors which is one of the limitations.

On the other hand, we added via points to prevent collisions with the tower. The arm keeps knocking down the tower as it moves to a coordinate because of the path that robot choose. So, we must force the arm to move in a path that's expected and prevent it to do unintended contacts with the tower. Each motion sequence was redesigned so the arm had to pass through a via point before moving down to pick up or drop a block.

We overcame the challenges and now the movement and sequence of the robot is more consistent.

Performance Analysis

We were satisfied with the overall structure of our code and the strategies we implemented, although the positional accuracy during construction of the second tower was not ideal. The finite state machine we implemented in our code allowed us to organize the robot's behavior into predictable stages and made it easy to refine individual coordinates and via-points. In practice, the FSM perfectly picks up the first, third, and fifth blocks of the tower, placed them in the designated drop zone, and consistently returned the arm to the neutral position between actions.

Our use of via-points improved the motion safety. It forces the arm to travel through intermediate heights before moving down, which prevent the arm knocking down the tower. Also, these predictable movements makes it convenient to coordinate with the human's movements. The robot followed a controlled, repeatable motion sequence, and no unexpected motion occurred. Which show an effective motion planning.

However, accuracy of dropping the block became a limiting factor. The tower that was built aren't straight and we need two manual interventions to fix it. We also reduced movement speed to improve the placement precision, but this increased total task time while we still can't built the perfect tower. Still the trade-off between speed and accuracy was worth it, slower speeds produced more stable and predictable end-effector behavior, while faster execution amplified servo overshoot and compounded drag-teaching inaccuracies.

Still, the robot's overall performance aligned well with our expectations given the mechanical and sensing constraints of the MechArm 270 Pi. It successfully deconstructs the tower and constructs a new tower. The arm executes this task following a collision-free path and demonstrates reliable integration of inverse and forward kinematics. Although there is a limitation in making a perfectly straight tower, the system is consistent with all the movements and demonstrated the effectiveness of our problem solving and motion planning strategies.

General Analysis

Overall Reflection

Lab 4 is a comprehensive integration of the skills and concepts developed throughout Labs 2 and 3. While earlier labs focused on individual components of robot operation, including drag teaching, forward kinematics, and inverse kinematics, Lab 4 required combining these elements into a real-life task-oriented system. Unlike previous labs where the objectives were primarily mathematical and model verification, as well as localized motion testing, Lab 4 emphasized more in precision manipulation, motion planning, and coordination under real-world constraints.

Challenge 1 emphasized precision and repeatability through the peg placement task, reinforcing the importance of accurate coordinate recording, stable inverse kinematics solutions, and careful motion sequencing. Even small errors in gripper positioning, angle, or height resulted in misalignment of placements. On the other hand, Challenge 2 introduced an incorporation of human-robot coordination. The robot was no longer operating by itself, but more as part of a coordinated system with actions between a human and an automated manipulator, which introduced timing constraints, dependency on external actions, and the need for precise synchronization.

In both cases, success depended on accurate kinematic modeling, stable inverse kinematics, and proper handling of intermediate waypoints. Both challenges also highlighted between theoretical correctness and physical feasibility. While it is true we have access to simulation and numerical solvers to provide us with correct solutions, real-world performance also has constraints from mechanical tolerances, joint limits, servo inaccuracies, and limited workspace. This reinforced that the approach to this problem is not purely a mathematical problem, but a physical systems engineering challenge.

Course Content Application

The successful completion of both challenges in Lab 4 was made possible through concepts applied and developed in Labs 2 and 3. Each prior lab contributed a distinct and essential layer to the final implementation rather than introducing new approaches.

From Lab 2, we developed fundamental hardware-level interaction with the MechArm 270 Pi through drag teaching and forward kinematics. The ability to manually guide the robot, record joint angles and end-effector coordinates, and store these values in CSV format became directly applicable in Lab 4. Both Challenges relied on these recorded datasets to establish precise pickup locations, insertion points, and block placement coordinates. This also helped us to use inverse kinematics to compute joint angles instead of commanding raw coordinates directly.

From Lab 3, we developed the mathematical framework necessary for accurate motion planning. The forward kinematics model using Denavit-Hartenberg parameters determined in Lab 2 was critical for verifying inverse kinematics outputs before executing movements on the physical robot because it computes the end-effector pose from joint angles. Furthermore, we used the full 6D pose-error optimization method over simplified inverse kinematics method because it helped minimized the full 6DOF error and produced consistent, physically accurate joint solutions, which also formed the backbone of both the peg placement and tower manipulation in Lab 4.

Process vs. Outcome

Although both challenges were completed successfully, the most important value of Lab 4 came from the development process rather than the final outcome alone. Many early failures were not caused by incorrect mathematics which we have already undergone and troubleshooted in previous labs, but by real-world factors such as timing, mechanical limitations, servo behavior, and unintended collisions. This showed that numerical correctness does not always translate to physical reliability.

Throughout both challenges, performance improved through iterative refinement. We introduced via points, adjusted motion speeds, modified gripper timing, and repeatedly recalibrated using the neutral position. These changes were driven by testing and failure analysis rather than by theory alone. In Challenge 1, small adjustments in insertion depth and speed determined whether a peg seated properly. In Challenge 2, redesigning the motion into staged pickup and drop phases using a finite-state approach was necessary to prevent tower collapse.

Overall, the success of Lab 4 depended more on systematic debugging, tuning, and motion planning than on any single correct equation. This reinforced that robotic system performance is achieved through process-driven engineering, not just task completion.

Lessons Learned

Lab 4 reinforced the idea that successful robotic systems depend as much on practical tuning and system integration as on mathematical correctness. While forward and inverse kinematics provided the foundation for motion planning, real-world accuracy ultimately depended on careful adjustment of approach paths, gripper timing, and execution speed. This showed that robotics is not only a theoretical process, but also a hands-on engineering process shaped by physical constraints.

We also learned that simple direct movements were often insufficient and unsafe, while the use of via points and staged trajectories significantly improved stability and repeatability. In Challenge 2, implementing a finite-state machine clarified the robot's behavior and made debugging far more manageable.

Finally, this lab highlighted the necessity of iterative testing and refinement. Early failures provided critical feedback that guided improvements in both challenges. Moving forward, we will apply these lessons by prioritizing system-level planning, structured control logic, and incremental validation when developing more complex robotic systems.

References

N/A. We consulted our previous Lab 2 and 3 submissions, Tawsif, the Lab 4 specification, and the pymycobot API reference.

Hours Spent & Feedback

We spent about 20 hours on this lab. Most of the time spent was tuning our coordinates for each robot movement. This was also the most frustrating part for us. One piece of feedback we have is having consistent peg setups. One type of wooden bench had plastic inserts that don't allow the peg to slide all the way into the slot. The other type had narrower pegs but they can slide all the way through the slot. Having consistent setups can help ensure that Challenge 1 strategies can apply to multiple peg bench setups. This lab helped us use tools we developed over the course of the quarter to solve open-ended challenges and navigate ambiguities. Solving these open-ended problems prepares us for similar challenges we might face in the real world.